

Machine Learning Report: Clustering

Gordon Wai Hin Kam

Task Description

The aim of this clustering task is to explore the unsupervised image clustering on MNIST dataset (70,000 handwritten digits, each 28×28 pixels) using K-Means and Gaussian Mixture Model (GMM).

Methods & Experiment

Data Preprocessing

I applied PCA to reduce the original 784 dimensional feature space into 50 components for achieving a feasible training time.

Scoring Metrics for Clustering Model

I created a helper function (`evaluate_score`) to return the Adjusted Random Index (ARI), Normalised Mutual Information (NMI) and Silhouette Score (SS) for each model.

Adjusted Random Index: measures how well the clustering assignments match the true label, correcting for chance agreement (i.e. some points will match just by random coincidence).

$$\text{ARI} = 1 \implies \text{perfect clustering}$$

$$\text{ARI} = 0 \implies \text{random clustering}$$

$$\text{ARI} = -1 \implies \text{worse than random clustering}$$

Normalised Mutual Information: measures the amount of shared information between the predicted clusters and the true labels.

$$\text{NMI} = 0 \implies \text{no mutual information}$$

$$\text{NMI} = 1 \implies \text{shares the same informations as the true labels}$$

Silhouette Score: evaluates the internal structure of a clustering by comparing:

- the distance between a sample and other points in the **same** cluster
- the distance between the sample and the points in the **nearest other** cluster

These metrics are combined with weights to evaluate the best clustering model for the MNIST dataset. Further details of how these metrics will be interpreted in the analysis section.

Building K-Means Model and Hyperparameters Selection

The K-Means clustering model was implemented using the `KMeans` class from the scikit-learn library. I performed a grid search cross-validation by exploring the number of clusters n_{cluster} and the number of initialisation n_{init} :

$$n_{\text{cluster}} \in \{5, 8, 10, 12\}, \quad n_{\text{init}} \in \{10, 20, 30\}$$

Building Gaussian Mixture Model and Hyperparameters Selection

The GMM was implemented using the `GaussianMixture` class from the scikit-learn library. I performed a grid search cross-validation by exploring the number of components $n_{\text{components}}$ and the covariance type `cov_type`:

$$n_{\text{components}} \in \{5, 8, 10, 12\}, \quad \text{cov_type} \in \{\text{full, tied, spherical, diag}\}$$

Visualisation

A custom visualisation function (`plot_clusters_grid`) was implemented to display the results of the best clustering models. In this function, I chose to display the first 5 images for each cluster by reshaping the input flattened image into 28×28 pixels.

Results & Analysis

Scoring Schemes

Each model is evaluated using ARI, NMI, and SS through the weighted scoring formula

$$\text{Combined Score} = w_{\text{ARI}} \cdot \text{ARI} + w_{\text{NMI}} \cdot \text{NMI} + w_{\text{SS}} \cdot \text{SS},$$

where I set $w_{\text{ARI}} = 0.5$, $w_{\text{NMI}} = 0.3$, and $w_{\text{SS}} = 0.2$.

I assigned the highest weighting to the ARI score because it provides the most important indication of how accurately the predicted clustering assignments correspond to the true labels. The reason of higher weighting on NMI than SS is that NMI also uses true labels to compare the same shared information, but SS only captures the distance between data points and clusters, it doesn't tell us whether the clusters correspond to the actual digit classes.

Best-Performing Models Across All Evaluation Metrics

The table below presents the best-performing model corresponding to each evaluation metric:

Metric	Model ID	Model	n_{clusters}	n_{init}	$n_{\text{components}}$	Cov. Type	Best Score
Best ARI	20	GMM	N/A	N/A	10	full	0.448
Best NMI	20	GMM	N/A	N/A	10	full	0.616
Best SS	3	K-Means	8	10	N/A	N/A	0.098
Best Combined Score	20	GMM	N/A	N/A	10	full	0.418

Table 1: Summary of the best-performing models.

The best-performing model for this dataset is a GMM with $n_{\text{components}} = 10$ and a full covariance matrix, achieving a best combined score of 0.418.

It makes sense that $n_{\text{components}} = 10$ is the optimal choice for components, since MNIST dataset naturally contains 10 digit classes (i.e. 0–9).

The full covariance type outperforms the other type because it captures complex handwriting variability (e.g. the writing width, rotation, and curvature of the digit). It provides the maximum flexibility for each component to learn its own covariance matrix and captures all

pairwise correlations between features, while the other types restrict the components by limiting the shape and orientation that they can represent.

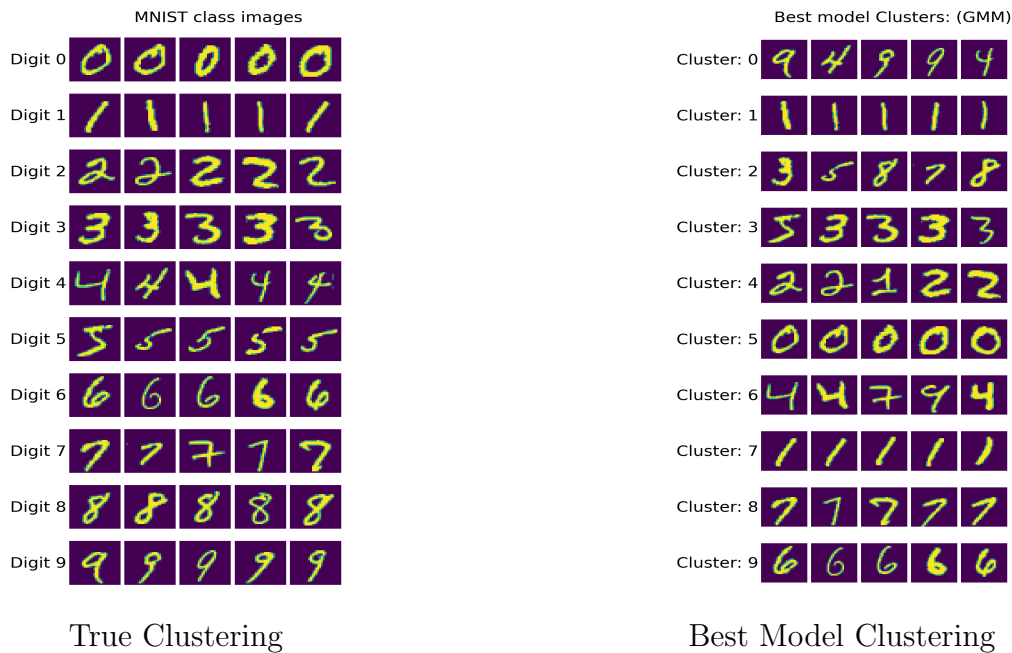


Figure 1: Comparison of the true and the best model clustering visualisations

For the Pure clusters, they have consistent shapes :

- **Digit 0** has a stable closed-loop structure that is rarely confused with any other digit.
- **Digit 1** is very simple with a straight vertical stroke.
- **Digit 6** has a circular body with a distinct tail on top.
- **Digit 7** has a sharp angular shape with a diagonal stroke and horizontal bar.

Reason for the impure cluster:

- **Overlapping and similar structure:** If we look at cluster 2, digits 3, 5 and 8 share an overlapping structure (i.e., the inverted 'c' shape) on the bottom half of their structures. Digits 4 and 9 often mix because both have a similar loop-like shape. Digit 4 has an open triangular loop, while 9 has a closed loop (as seen in cluster 0).
- **Large variance of handwritings:** The large variation of handwriting styles due to their rotation, curvature can easily be mixed up during clustering. For instance, digit 1 was misclustered into cluster 4 because writing it with a horizontal stroke makes it resemble digit 2.

Self-Reflection

Through experimenting with K-Means, GMM, I gained a deeper understanding of the strengths and limitations of unsupervised learning and improved my ability to interpret clustering outcomes.